



IASI Quarto

Guía de usuario

IASI Project

Table of contents

1	IASI Quarto	4
2	Introducción	5
3	Conceptos	6
3.1	Las ideas básicas	6
3.2	Cuatro tipos de proyecto	6
3.3	Tipo y estrategia no son lo mismo	7
3.4	Tres niveles de materialización	7
4	Instalación	8
4.1	Requisitos	8
4.2	Instalar el paquete	8
4.3	Primera comprobación	9
5	Proyectos IASI	10
5.1	Marcar un proyecto	10
5.2	Una guía Quarto	10
5.3	Un sitio web	11
5.4	Un paquete R	11
5.5	Una raíz común	11
6	Publicaciones	13
6.1	Las cuatro estrategias de libro	13
6.2	Estrategia regular	13
6.3	Estrategia structured	14
6.4	Estrategia parted	14
6.5	Estrategia direct	15
6.6	Front matter y back matter	16
6.7	Landing page HTML	16
7	Formatos	17
7.1	Perfiles disponibles	17
7.2	Declarar perfiles	17
7.3	Construir un formato	18
7.4	GitBook	18

7.5	HTML y exportaciones	18
8	Operaciones	19
8.1	<code>validate()</code>	19
8.2	<code>build()</code>	19
8.3	<code>publish()</code>	20
8.4	<code>release()</code>	21
8.5	Flujo recomendado	21
9	Trabajar con varios proyectos	23
9.1	Descubrimiento desde una carpeta	23
9.2	Trabajar solo con uno	24
9.3	Publicación de varios proyectos	24
9.4	Configuración común	24
10	Configuración	25
10.1	<code>_iasi.yml</code>	25
10.2	<code>publication</code>	25
10.3	<code>paths</code>	26
10.4	Configuración heredada	26
10.5	<code>exclude</code>	26
10.6	<code>_quarto.yml</code>	27
11	Problemas frecuentes	28
11.1	El proyecto no se reconoce	28
11.2	Tengo <code>_iasi.yml</code> y <code>.iasi.yml</code>	28
11.3	Un formato no se construye	28
11.4	<code>build()</code> vuelve a construir	29
11.5	<code>publish()</code> dice que no hay nada que hacer	29
11.6	<code>publish()</code> no construye mis cambios	29
11.7	<code>release()</code> no genera una publicación nueva	29
11.8	No aparece una carpeta en un libro <code>numbered</code>	30
11.9	Quiero desplegar a GitHub Pages u otro servicio	30

1 IASI Quarto

Guía de usuario

2 Introducción

`iasi.quarto` es un paquete R que ayuda a **validar, construir, preparar para publicación y reunir entregables** de proyectos IASI basados en Quarto y de paquetes R.

Su objetivo es que el usuario pueda concentrarse en el contenido y en la configuración del proyecto sin tener que reconstruir manualmente la navegación, los artefactos auxiliares o el proceso de publicación cada vez.

La API pública es pequeña:

```
validate()
build()
publish()
release()
```

Cada operación tiene una responsabilidad distinta:

```
validate → comprobar
build    → construir
publish  → preparar una publicación local
release  → reunir entregables locales
```

`iasi.quarto` **no despliega a servicios externos**. Esa responsabilidad corresponde a herramientas de nivel superior, como `iasi-dev`.

Esta guía describe el uso de `iasi.quarto` 0.10.2. No necesita conocer su arquitectura interna para seguirla.

3 Conceptos

3.1 Las ideas básicas

Antes de empezar conviene distinguir unas pocas palabras que aparecen continuamente.

Término	Significado
Proyecto IASI	Carpeta que contiene <code>_iasi.yml</code> o <code>.iasi.yml</code> .
Proyecto Quarto	Proyecto que Quarto reconoce mediante <code>_quarto.yml</code> .
Tipo IASI	Indica qué clase de proyecto es para IASI: <code>repository</code> , <code>guide</code> , <code>web</code> o <code>r-package</code> .
Estrategia	Indica cómo interpretar la organización editorial de un libro.
Formato / perfil	Salida que se quiere construir, por ejemplo <code>html</code> , <code>pdf</code> , <code>epub</code> o <code>git</code> .
Build	Construcción de los artefactos en <code>_outputs/</code> , o de los paquetes R.
Publish	Preparación local de una publicación a partir de <code>_outputs/</code> .
Release	Reunión de los entregables locales ya contruidos o publicados.

3.2 Cuatro tipos de proyecto

Tipo	Para qué sirve	Build	Publish	Release
<code>repository</code>	Raíz o contenedor común			
<code>guide</code>	Libro o guía Quarto			
<code>web</code>	Sitio web Quarto			
<code>r-package</code>	Paquete R			

Un proyecto **repository** puede proporcionar configuración común a proyectos situados debajo, pero no se construye como entregable.

3.3 Tipo y estrategia no son lo mismo

Un **guide** indica **qué es el proyecto**.

La estrategia (**regular**, **structured**, **parted** o **direct**) indica **cómo está organizado editorialmente el libro**.

Por ejemplo, dos proyectos pueden ser ambos **guide** y utilizar estrategias distintas sin dejar de ser guías.

3.4 Tres niveles de materialización

Para una publicación Quarto:

```
fuentes
  ↓ build()
_outputs/
  ↓ publish()
_publish/
  ↓ release()
release/
```

Los tres directorios representan estados diferentes. No son copias intercambiables.

4 Instalación

4.1 Requisitos

Para utilizar `iasi.quarto` necesita:

- R 4.2 o posterior;
- Quarto;
- Git si instala el paquete directamente desde GitHub;
- las herramientas adicionales que exija cada formato.

Por ejemplo, un PDF puede necesitar una instalación TeX funcional.

Compruebe R y Quarto:

```
R --version  
quarto --version
```

4.2 Instalar el paquete

Puede instalarlo desde GitHub con `remotes`:

```
install.packages("remotes")  
remotes::install_github("iasi-org/iasi-quarto")
```

Después:

```
library(iasi.quarto)
```

Compruebe la versión instalada:

```
packageVersion("iasi.quarto")
```

4.3 Primera comprobación

Sitúese en un proyecto IASI y ejecute:

```
validate()
```

`validate()` comprueba el proyecto sin construirlo.

Si trabaja desde otra carpeta puede indicar la ruta:

```
validate(path = "docs/01-user-guide")
```

5 Proyectos IASI

5.1 Marcar un proyecto

Una carpeta se convierte en proyecto IASI al contener **uno** de estos ficheros:

```
_iasi.yml  
.iasi.yml
```

No utilice ambos en la misma carpeta.

La configuración mínima indica el tipo:

```
type: guide
```

o, por ejemplo:

```
type: r-package
```

5.2 Una guía Quarto

Un **guide** necesita además su configuración Quarto:

```
# _quarto.yml  
project:  
  type: book
```

y su configuración IASI:

```
# _iasi.yml  
type: guide  
publication:  
  strategy: regular
```

Para los libros, si no indica otra cosa, IASI utiliza estos valores:

```
strategy      = regular
front-matter  = front-matter
content-dir   = chapters
back-matter   = back-matter
numbered      = true
```

5.3 Un sitio web

```
# _iasi.yml
type: web
publication:
  strategy: regular
```

```
# _quarto.yml
project:
  type: website
```

Los websites utilizan la estrategia `regular`.

5.4 Un paquete R

```
type: r-package
```

El proyecto debe tener la estructura normal de un paquete R, incluido `DESCRIPTION`.

5.5 Una raíz común

Una carpeta que agrupa varios proyectos puede declararse:

```
type: repository
```

La raíz puede contener valores comunes, por ejemplo un directorio compartido de release:

```
type: repository

paths:
  release: release
```

Los proyectos hijos pueden omitir valores que ya están definidos en una carpeta superior. Sus propios valores siempre tienen prioridad.

6 Publicaciones

6.1 Las cuatro estrategias de libro

Los libros pueden utilizar cuatro estrategias.

Estrategia	Úsela cuando...
<code>regular</code>	varios <code>.qmd</code> de una carpeta forman una única unidad editorial;
<code>structured</code>	quiere representar explícitamente Partes que contienen Capítulos;
<code>parted</code>	necesita Partes con una combinación más rica de capítulos directos y carpetas de capítulo;
<code>direct</code>	cada <code>.qmd</code> ya es un capítulo completo.

Un Website utiliza `regular`.

La estrategia es una decisión **editorial**. No depende de si después construye HTML, PDF o EPUB.

6.2 Estrategia regular

`regular` es la estrategia predeterminada para libros.

Una carpeta representa una unidad editorial y los `.qmd` que contiene se combinan en el `index.qmd` generado de esa carpeta.

```
chapters/  
  01-introduction/  
    front-matter.quarto  
    01-purpose.qmd  
    02-scope.qmd
```

`front-matter.quarto` puede definir, por ejemplo:

```
---  
title: "Introducción"  
title-block-style: none  
---
```

IASI genera el `index.qmd`. Edite los documentos fuente y `front-matter.quarto`, no el `index.qmd` generado.

6.3 Estrategia structured

`structured` representa directamente una jerarquía **Parte** → **Capítulo**.

```
chapters/  
  20-methodologies/  
    index.qmd  
    01-history.qmd  
    02-current.qmd
```

La carpeta `20-methodologies` representa la Parte.

Su `index.qmd` presenta esa Parte y los otros `.qmd` son Capítulos.

En esta estrategia el `index.qmd` es fuente y debe existir:

```
publication:  
  strategy: structured  
  content-dir: chapters  
  numbered: true
```

6.4 Estrategia parted

`parted` permite construir una Parte más rica.

Cada carpeta principal representa una Parte y debe contener `index.qmd`. Dentro puede mezclar:

- `intro.qmd`;
- capítulos `.qmd` directos;
- carpetas que representan capítulos y que tienen su propio `index.qmd`;
- documentos dentro de esas carpetas de capítulo.

Ejemplo:

```
chapters/  
  20-engineering/  
    index.qmd  
    intro.qmd  
    01-principles.qmd  
    02-lifecycle/  
      index.qmd  
      01-build.qmd  
      02-publish.qmd
```

En `_iasi.yml`:

```
publication:  
  strategy: parted  
  content-dir: chapters  
  numbered: true
```

Si `numbered: true`, los capítulos directos y las carpetas de capítulo utilizan prefijos numéricos y no deben repetir el mismo número dentro de una Parte.

Use `parted` cuando `structured` resulte demasiado plano pero quiera conservar una jerarquía editorial explícita.

6.5 Estrategia direct

`direct` trata cada documento `.qmd` como un capítulo.

```
chapters/  
  01-concepts/  
    01-model.qmd  
    02-context.qmd  
  02-usage/  
    01-build.qmd  
    02-publish.qmd
```

El libro contiene cuatro capítulos consecutivos. Las carpetas ayudan a organizar los fuentes, pero no crean por sí mismas Partes.

```
publication:
  strategy: direct
```

6.6 Front matter y back matter

Un libro puede reservar carpetas para contenido anterior y posterior al cuerpo principal:

```
publication:
  front-matter: front-matter
  content-dir: chapters
  back-matter: back-matter
```

Estas carpetas se integran en la estructura del libro al construirlo.

La estrategia elegida sigue determinando cómo se interpreta el contenido principal.

6.7 Landing page HTML

Un libro puede pedir que su salida HTML se trate como Website:

```
publication:
  html:
    landing-page: true
```

Esta opción solo es válida para proyectos Quarto de tipo `book`.

7 Formatos

7.1 Perfiles disponibles

IASI Quarto construye perfiles Quarto declarados por el proyecto para los que dispone de soporte.

Los perfiles habituales son:

Perfil	Resultado
<code>web</code>	Website
<code>html</code>	Libro HTML
<code>pdf</code>	PDF
<code>pdfua</code>	PDF/UA mediante Typst
<code>epub</code>	EPUB
<code>docx</code>	DOCX
<code>odt</code>	ODT
<code>git</code>	Estructura preparada para GitBook

Un Website utiliza el perfil `web`. Una guía o libro utiliza `html` para su salida HTML.

7.2 Declarar perfiles

Un proyecto puede declarar sus perfiles en `_quarto.yml`:

```
profile:
  default: html
  group:
    - [html, pdf, epub]
```

y configurar cada uno en su fichero:

```
_quarto-html.yml  
_quarto-pdf.yml  
_quarto-epub.yml
```

7.3 Construir un formato

```
build(format = "html")
```

Varios formatos:

```
build(format = c("html", "pdf"))
```

Todos los perfiles declarados que tengan soporte:

```
build()
```

Si solicita un perfil que el proyecto no ha declarado, IASI Quarto lo ignora con un aviso.

7.4 GitBook

El perfil `git` genera la estructura que espera GitBook, incluido:

```
README.md  
SUMMARY.md  
.gitbook.yaml
```

7.5 HTML y exportaciones

Cuando se construye HTML junto con otros formatos, IASI Quarto puede generar la información necesaria para que la publicación HTML ofrezca enlaces a PDF, EPUB, DOCX, ODT o GitBook disponibles.

8 Operaciones

8.1 `validate()`

Use `validate()` para comprobar que los proyectos están correctamente configurados:

```
validate()
```

O desde otra ruta:

```
validate(path = "docs/01-user-guide")
```

Para publicaciones comprueba, entre otras cosas, la estructura editorial y la configuración.

Para paquetes R comprueba que el proyecto tiene los elementos necesarios, como DESCRIPTION.

No construye artefactos.

8.2 `build()`

`build()` **construye siempre que se invoca.**

```
build()
```

Para una publicación Quarto deja los resultados en:

```
_outputs/
```

separados por perfil:

```
_outputs/  
  html/  
  pdf/  
  epub/
```

Durante la edición es habitual construir solo lo que se está revisando:

```
build(format = "html")
```

Para un Website:

```
build(format = "web")
```

Para un `r-package`, `build()` ejecuta la construcción del paquete fuente y del paquete binario soportado por la plataforma.

8.3 `publish()`

`publish()` no vuelve a renderizar los fuentes.

Toma exclusivamente lo que ya existe en:

```
_outputs/
```

y prepara una publicación local:

```
_publish/
```

```
publish()
```

Antes de repetir el trabajo compara el contenido construido con la última publicación. Si no ha cambiado, puede terminar sin rehacer `_publish/`.

Para obligar a repetir la publicación:

```
publish(force = TRUE)
```

La nueva publicación se prepara primero en un directorio temporal. La publicación anterior no se sustituye hasta que la nueva está completa.

En una publicación con HTML y PDF puede quedar, por ejemplo:

```
_publish/  
  index.html  
  chapters/  
  pdf/  
    guide.pdf
```

El contenido pensado para navegador ocupa la raíz y los formatos alternativos quedan como exportaciones.

8.4 `release()`

`release()` reúne los entregables locales:

```
release()
```

No renderiza de nuevo y no despliega a Internet.

Para una guía o website toma el contenido ya preparado por `publish()`.

Para un paquete R recoge los artefactos construidos por `build()`.

El destino predeterminado es:

```
release/
```

Si varios proyectos comparten un `paths.release` definido en una carpeta superior, una guía se coloca en un subdirectorío derivado del nombre del proyecto. Los prefijos numéricos organizativos se eliminan:

```
01-user-guide → release/user-guide/  
02-developer-guide → release/developer-guide/
```

8.5 Flujo recomendado

Durante la edición:

```
build(format = "html")
```

Cuando quiera construir todos los perfiles declarados:

```
build()
```

Después:

```
publish()  
release()
```

En resumen:

```
validate  → comprobar  
build     → construir  
publish   → preparar  
release   → reunir
```

9 Trabajar con varios proyectos

9.1 Descubrimiento desde una carpeta

El argumento `path` define desde dónde buscar proyectos IASI.

```
build(path = ".")
```

IASI Quarto recorre esa carpeta y sus subcarpetas y encuentra proyectos marcados con `_iasi.yml` o `.iasi.yml`.

Los directorios intermedios no necesitan ser proyectos IASI.

Ejemplo:

```
iasi-quarto/  
  _iasi.yml                # type: repository  
  docs/  
    01-user-guide/  
      _iasi.yml  
      _quarto.yml  
    02-developer-guide/  
      _iasi.yml  
      _quarto.yml  
    03-technical-guide/  
      _iasi.yml  
      _quarto.yml  
  software/  
    iasi-quarto/  
      _iasi.yml  
      DESCRIPTION
```

Desde la raíz:

```
validate(path = ".")  
build(path = ".")  
release(path = ".")
```

se aplican a los proyectos para los que cada operación tiene sentido.

9.2 Trabajar solo con uno

No necesita un selector especial. Indique su carpeta:

```
build(path = "docs/01-user-guide", format = "html")
```

9.3 Publicación de varios proyectos

`publish(path = ".")` puede preparar varias publicaciones encontradas bajo esa raíz, pero **cada proyecto conserva su propio destino de publicación.**

Con los valores predeterminados:

```
docs/01-user-guide/_publish/  
docs/02-developer-guide/_publish/  
docs/03-technical-guide/_publish/
```

No existe un único `_publish/` de multiproyecto.

La reunión de entregables corresponde a `release()`.

9.4 Configuración común

Una raíz `repository` puede aportar defaults:

```
type: repository  
  
paths:  
  release: release
```

Los proyectos hijos heredan los valores que no hayan definido por sí mismos.

Esto permite, por ejemplo, reunir varias guías bajo un único árbol `release/` sin obligarlas a compartir `_publish/`.

10 Configuración

10.1 `_iasi.yml`

El fichero `_iasi.yml` describe las decisiones propias de IASI.

Ejemplo de una guía:

```
type: guide

publication:
  strategy: parted
  content-dir: chapters
  numbered: true

paths:
  publish: _publish
  release: release
```

10.2 publication

Para libros puede utilizar:

```
publication:
  strategy: regular
  front-matter: front-matter
  content-dir: chapters
  back-matter: back-matter
  numbered: true
  html:
    landing-page: false
```

Los valores predeterminados de un libro son `regular`, `front-matter`, `chapters`, `back-matter` y `numbered: true`.

10.3 paths

Los paths principales son:

```
paths:
  publish: _publish
  release: release
```

La clave correcta es:

```
paths.release
```

No utilice la forma antigua `paths.releases`.

10.4 Configuración heredada

Si una carpeta superior también tiene configuración IASI, sus valores sirven como defaults para los proyectos situados debajo.

Ejemplo:

```
repository/
  _iasi.yml
  docs/
    01-user-guide/
      _iasi.yml
```

Si la raíz define `paths.release` y la guía no lo redefine, la guía utiliza ese valor.

Si la guía sí lo define, gana el valor local.

10.5 exclude

Puede excluir nombres que no deben participar en la estructura editorial:

```
exclude:
- drafts
- archive
```

10.6 `_quarto.yml`

IASI Quarto complementa a Quarto, no lo sustituye.

Los perfiles, formatos, tema, título, filtros y demás opciones propias de Quarto siguen configurándose mediante `_quarto.yml` y sus perfiles `_quarto-*.yml`.

11 Problemas frecuentes

11.1 El proyecto no se reconoce

Compruebe que la carpeta contiene exactamente uno de estos ficheros:

```
_iasi.yml  
.iasi.yml
```

Para una guía o website también debe existir `_quarto.yml`.

Para un paquete R debe existir `DESCRIPTION`.

Pruebe:

```
validate(path = "ruta/del/proyecto")
```

11.2 Tengo `_iasi.yml` y `.iasi.yml`

No utilice los dos en la misma carpeta. La configuración sería ambigua.

11.3 Un formato no se construye

Compruebe dos cosas:

1. que el perfil está declarado por el proyecto;
2. que dispone de las herramientas externas que necesita.

Ejemplo:

```
_quarto-pdf.yml
```

Para un Website recuerde que el perfil IASI esperado es `web`.

11.4 build() vuelve a construir

Es el comportamiento esperado.

Una llamada a:

```
build()
```

significa **construir ahora**.

11.5 publish() dice que no hay nada que hacer

Si `_outputs/` no ha cambiado desde la última publicación, `publish()` puede omitir el trabajo.

Para repetirlo expresamente:

```
publish(force = TRUE)
```

11.6 publish() no construye mis cambios

`publish()` no renderiza.

Primero:

```
build()
```

y después:

```
publish()
```

11.7 release() no genera una publicación nueva

También es intencionado.

`release()` reúne lo que ya fue construido o publicado. Para una guía:

```
build()  
publish()  
release()
```

11.8 No aparece una carpeta en un libro `numbered`

Cuando `numbered: true`, las carpetas y documentos que participan en el orden editorial deben seguir la convención numérica que espera la estrategia.

Por ejemplo:

```
01-introduction
02-architecture
03-usage
```

En `parted`, no repita el mismo prefijo numérico entre un capítulo directo y una carpeta de capítulo dentro de la misma Parte.

11.9 Quiero desplegar a GitHub Pages u otro servicio

Eso queda fuera de `iasi.quarto`.

`iasi.quarto` termina en `release()`. Utilice la herramienta responsable del despliegue, por ejemplo `iasi-dev`.